

RAG Scan Stack Complete Platform Guide

Contents

1	RAG Scan Stack - User Guide & Features Overview	3
1.1	Table of Contents	3
1.2	1. Platform Introduction	3
1.2.1	What is RAG Scan Stack?	3
1.2.2	Key Value Propositions	3
1.3	2. Dashboard Overview	4
1.3.1	Main Navigation Areas	4
1.3.2	Dashboard Status Cards	4
1.4	3. Engagement Management Workflow	6
1.4.1	Step 1: Create New Engagement	6
1.4.2	Step 2: Engagement Timeline & Phases	6
1.4.3	Step 3: Status Tracking	7
1.5	4. Scope Definition & Intelligence	7
1.5.1	Step 1: Define Target Scope	7
1.5.2	Step 2: Automated Intelligence Gathering	8
1.5.3	Step 3: Scope Validation & Approval	8
1.6	5. AI-Powered Scanning Workflow	10
1.6.1	Step 1: AI Agent Overview	10
1.6.2	Step 2: Automated Scan Pipeline	10
1.6.3	Step 3: Manual Scan Initiation	10
1.7	6. Findings Management	11
1.7.1	Step 1: Findings Explorer Interface	11
1.7.2	Step 2: Finding Details & Evidence	11
1.7.3	Step 3: Workflow Management	12
1.8	7. Node & Tunnel Management	12
1.8.1	Step 1: Remote Access Overview	12
1.8.2	Step 2: WireGuard Management	15
1.8.3	Step 3: SSH Tunnel Operations	15
1.9	8. Reporting & Export	16
1.9.1	Step 1: Report Generation Options	16
1.9.2	Step 2: Customizable Report Content	16
1.9.3	Step 3: Integration & Distribution	17
1.10	9. Advanced Features	17
1.10.1	Delta Analysis & Trending	17
1.10.2	API Security Testing	17
1.10.3	Cloud Security Posture	18
1.10.4	Credential Testing & Management	18
1.11	Next Steps	18
1.11.1	For Pentesters & Red Teams	18
1.11.2	For Security Managers	18
1.11.3	Training & Support Resources	18
1.12	Conclusion	18

1.13	Table of Contents	19
1.14	1. Platform Administration Overview	19
	1.14.1 Administrative Responsibilities	19
	1.14.2 Admin Access Methods	19
1.15	2. Service Health Monitoring	19
	1.15.1 Health Dashboard Overview	19
	1.15.2 Core Service Monitoring	21
	1.15.3 Scanner Service Monitoring	21
1.16	3. Container & Service Management	21
	1.16.1 Service Control Interface	21
	1.16.2 Container Management	23
	1.16.3 Health Check Configuration	23
1.17	4. Performance Monitoring & Metrics	24
	1.17.1 Resource Monitoring	24
	1.17.2 Performance Dashboards	25
	1.17.3 Alerting & Notifications	25
1.18	5. Database Administration	26
	1.18.1 Database Overview	26
	1.18.2 Database Health Monitoring	26
	1.18.3 Database Maintenance	26
	1.18.4 Schema Management	27
1.19	6. User Management & Security	27
	1.19.1 Authentication & Authorization	27
	1.19.2 Security Configuration	27
	1.19.3 Audit & Compliance	28
1.20	7. Backup & Recovery	28
	1.20.1 Backup Strategy	28
	1.20.2 Recovery Procedures	28
	1.20.3 Disaster Recovery	29
1.21	8. Troubleshooting Guide	29
	1.21.1 Common Issues & Solutions	29
	1.21.2 Performance Issues	30
	1.21.3 Log Analysis	30
1.22	9. Maintenance Procedures	30
	1.22.1 Regular Maintenance Tasks	30
	1.22.2 Update Management	31
	1.22.3 Monitoring & Alerting Maintenance	31
1.23	Emergency Procedures	32
	1.23.1 Critical System Failure	32
	1.23.2 Security Incident Response	32
1.24	Conclusion	32
1.25	System Overview	32
	1.25.1 Core Principle	32
1.26	Basic Architecture	33
1.27	Technology Stack	33
	1.27.1 Core Components	33
	1.27.2 Security Scanners (Each in separate container)	33
1.28	API-First Design	33
	1.28.1 Everything Has an API Endpoint	33
	1.28.2 Direct Scanner APIs	34
1.29	MCP Tool Integration	34
	1.29.1 What is MCP?	34
	1.29.2 Available MCP Tools (26 total across 6 servers)	34
	1.29.3 MCP Server Endpoints	34

1.30	Data Flow	34
1.30.1	Simple Scan Workflow	34
1.30.2	Database Schema (Simplified)	35
1.31	Remote Access & Tunneling	35
1.31.1	SSH Tunnels	35
1.31.2	WireGuard VPN	35
1.32	Deployment	35
1.32.1	Single Host Deployment	35
1.32.2	Environment Requirements	35
1.33	Security Model	35
1.33.1	Container Isolation	35
1.33.2	API Security	36
1.33.3	Data Protection	36
1.34	Integration Points	36
1.34.1	Import/Export	36
1.34.2	External APIs	36
1.35	Monitoring & Health	36
1.35.1	Service Health	36
1.35.2	Logging	36
1.36	Summary	36

1 RAG Scan Stack - User Guide & Features Overview

Comprehensive Penetration Testing & Red Team Workflow Platform

1.1 Table of Contents

1. Platform Introduction
 2. Getting Started - Dashboard Overview
 3. Engagement Management
 4. Scope Definition & Intelligence
 5. AI-Powered Scanning
 6. Findings Management
 7. Node & Tunnel Management
 8. Reporting & Export
 9. Advanced Features
-

1.2 1. Platform Introduction

1.2.1 What is RAG Scan Stack?

- **Comprehensive Security Testing Platform** for authorized penetration testing and red team operations
- **AI-Enhanced Workflow** with automated scanning, intelligent findings correlation, and multi-agent coordination
- **Centralized Data Collection** from 20+ security tools with unified findings management
- **Advanced Tunneling** with SSH, WireGuard, and C2 framework support for remote assessments

1.2.2 Key Value Propositions

- **[SUCCESS] Workflow Acceleration:** Reduce manual coordination between tools by 80%

- [SUCCESS] **AI-Driven Intelligence:** Automated reconnaissance, scan recommendations, and finding correlation
- [SUCCESS] **Unified Findings Management:** Single source of truth for all security findings
- [SUCCESS] **Remote Assessment Ready:** Built-in tunneling and proxy support for complex network environments

1.3 2. Dashboard Overview

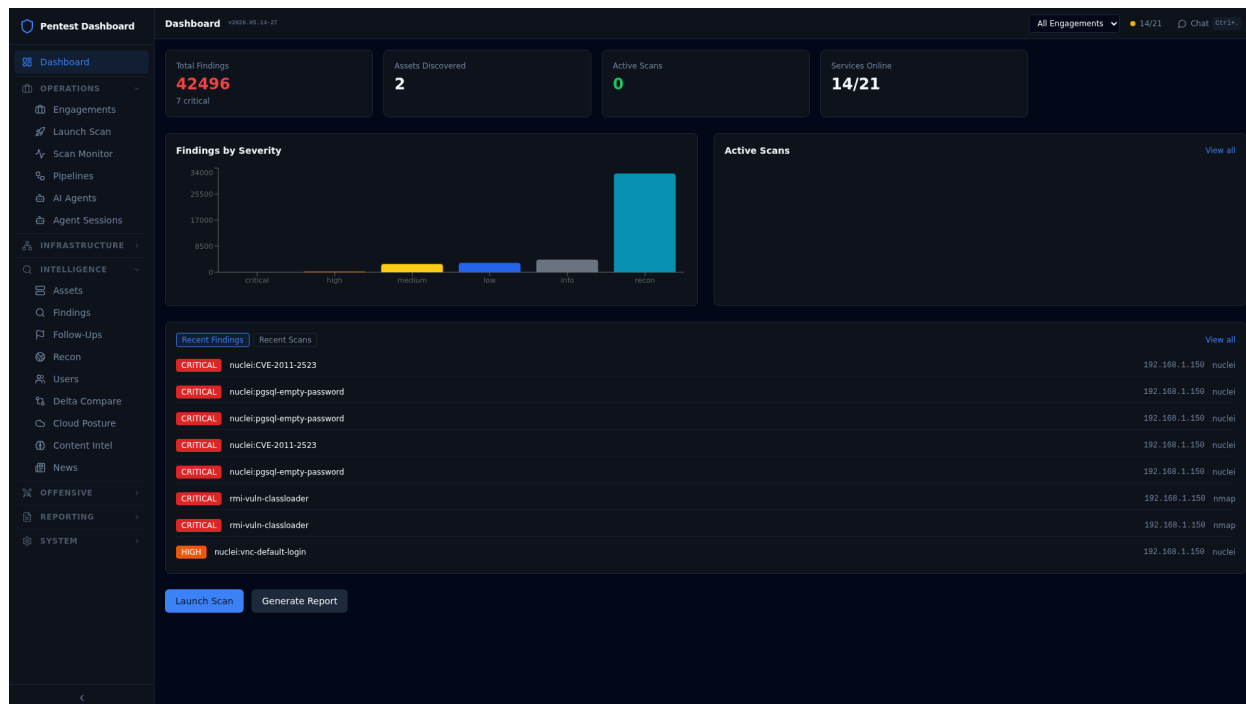


Figure 1: Dashboard Overview

1.3.1 Main Navigation Areas

- **Dashboard**: Real-time status, scan activity, and quick actions
- [TARGET] **Engagements**: Project management and scope tracking
- [SEARCH] **Findings**: Comprehensive vulnerability and findings explorer
- **Assets**: Network discovery and asset intelligence
- **AI Agents**: Multi-agent automation and scan orchestration
- **Nodes**: Remote access and tunnel management
- **Settings**: Configuration and system management

1.3.2 Dashboard Status Cards

- **System Health**: Service status and performance metrics
- **Active Scans**: Real-time scan progress and queue status
- **Recent Findings**: Latest discoveries with severity indicators
- **Tunnel Status**: Remote node connectivity and proxy availability

Figure 2: Dashboard Overview

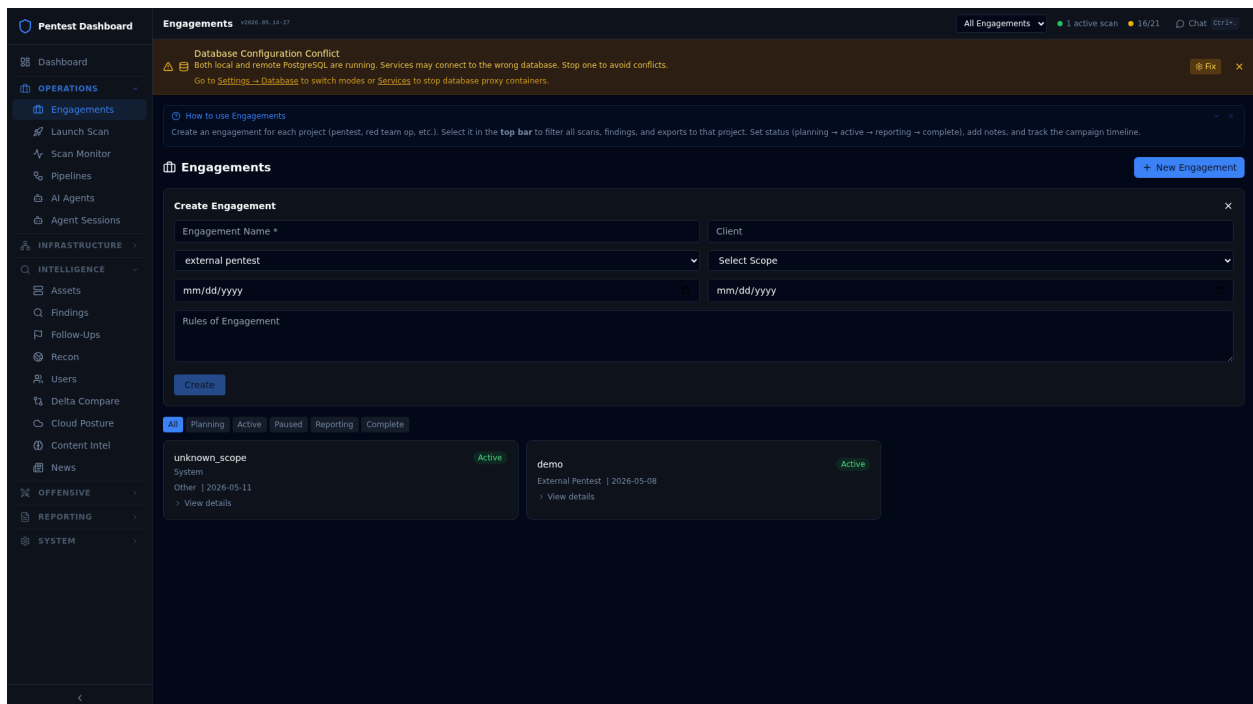


Figure 3: Engagements Management

Figure 4: Engagement Management

1.4 3. Engagement Management Workflow

1.4.1 Step 1: Create New Engagement

1.4.1.1 Navigation

1. Click **Engagements** in main navigation
2. Click **+ New Engagement** button
3. Fill engagement details:

1.4.1.2 Required Fields

Engagement Name: "Acme Corp Assessment 2026"
Client: "Acme Corporation"
Start Date: "2026-05-15"
End Date: "2026-05-30"
Engagement Type: "Internal Network Assessment"
Status: "Planning"

1.4.1.3 Optional Configuration

- **Scope Notes:** Internal network ranges and critical systems
- **Rules of Engagement:** Approved testing windows and restrictions
- **Emergency Contacts:** Client SPOC and escalation procedures
- **Testing Methodology:** OWASP, NIST, Custom framework selection

1.4.2 Step 2: Engagement Timeline & Phases

- **Planning Phase:** Scope definition and approval
- **Active Phase:** Scanning and manual testing
- **Reporting Phase:** Findings compilation and documentation

- **Complete Phase:** Deliverable submission and closure

1.4.3 Step 3: Status Tracking

- **Real-time progress indicators** for each engagement phase
 - **Automated timeline events** from scan completions and findings
 - **Manual milestone tracking** for client meetings and deliverables
-

1.5 4. Scope Definition & Intelligence



Figure 5: Scope Intelligence

1.5.1 Step 1: Define Target Scope

1.5.1.1 Manual Target Entry

Target Types Supported:

- IP Addresses: 192.168.1.0/24, 10.0.0.1-10.0.0.50
- Hostnames: mail.acme.com, *.internal.acme.com
- URLs: https://app.acme.com, http://admin.acme.local
- Domain Names: acme.com, subsidiary.net

1.5.1.2 Bulk Import Options

- **CSV Upload:** Structured target lists with metadata
- **Text Import:** Line-separated targets with auto-detection
- **API Integration:** Import from external asset management systems
- **Previous Engagement:** Clone scope from historical assessments

Figure 6: Recon Explorer

1.5.2 Step 2: Automated Intelligence Gathering

1.5.2.1 OSINT Collection

- **Domain Intelligence:** DNS records, subdomains, certificate transparency
- **Cloud Asset Discovery:** AWS, Azure, GCP resource enumeration
- **Social Engineering Intel:** Employee enumeration, email patterns
- **Technology Stack Detection:** Framework fingerprinting and version detection

1.5.2.2 Passive Reconnaissance

Tools Integrated:

Subfinder - Subdomain enumeration
DNSx - DNS resolution and validation
ASNmap - IP range and ownership mapping
Amass - Comprehensive asset discovery
Cloudlist - Cloud provider asset enumeration

1.5.3 Step 3: Scope Validation & Approval

- **Asset verification** with client-provided scope documentation
 - **Out-of-scope flagging** for discovered assets outside testing boundaries
 - **Approval workflow** with client stakeholder sign-off
 - **Change management** for scope modifications during testing
-

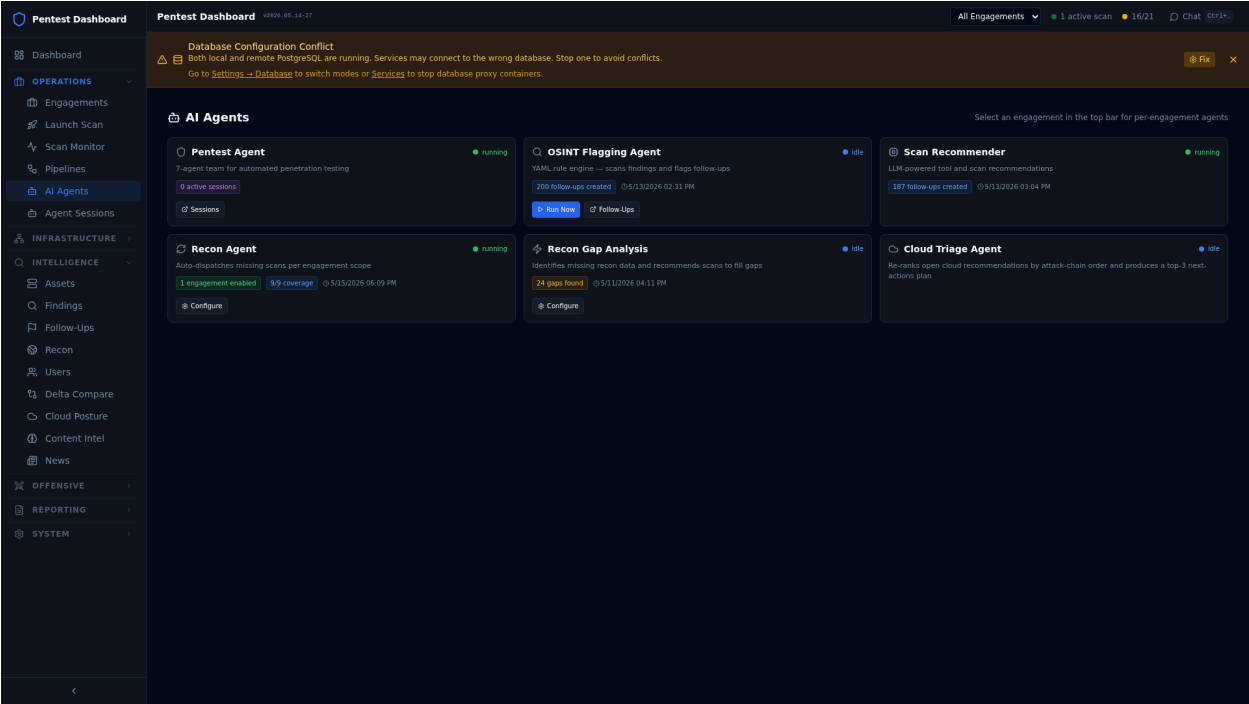


Figure 7: AI Agents Dashboard

Figure 8: AI Agents

1.6 5. AI-Powered Scanning Workflow

1.6.1 Step 1: AI Agent Overview

1.6.1.1 Multi-Agent Architecture

- **** Orchestrator Agent****: Coordinates scanning strategy and resource allocation
- **[SEARCH] Recon Agent**: Automated reconnaissance and target discovery
- **** Scanning Agent****: Vulnerability assessment and service enumeration
- **[FAST] Exploit Agent**: Safe exploit validation and proof-of-concept development
- **[ANALYTICS] Analysis Agent**: Finding correlation and risk assessment

1.6.2 Step 2: Automated Scan Pipeline

1.6.2.1 Phase 1: Discovery & Reconnaissance

Scan Sequence:

1. Port Discovery (Masscan/Nmap)
 - + TCP: 1-65535 (adaptive timing)
 - + UDP: Top 1000 ports
2. Service Enumeration
 - + Banner grabbing and fingerprinting
 - + TLS/SSL configuration analysis
 - + HTTP service discovery
3. Technology Detection
 - + Framework identification
 - + Version enumeration
 - + Security header analysis

1.6.2.2 Phase 2: Vulnerability Assessment

Scanning Tools:

Nuclei: 8000+ templates for known vulnerabilities
ZAP: Dynamic web application scanning
Playwright: Modern JavaScript application testing
Custom Scripts: Organization-specific checks

1.6.2.3 Phase 3: Intelligence Correlation

- **Cross-reference findings** across multiple tools
- **Risk scoring** based on exploitability and business impact
- **Attack path mapping** for lateral movement scenarios
- **False positive filtering** using AI-powered validation

1.6.3 Step 3: Manual Scan Initiation

1.6.3.1 Quick Launch Options

- **[LAUNCH] Full Pipeline**: Complete end-to-end assessment
- **[TARGET] Targeted Scan**: Specific service or vulnerability checks
- **** Delta Scan****: Changes since last assessment
- **** Emergency Scan****: Critical vulnerability validation

1.6.3.2 Custom Scan Configuration

Scan Parameters:

- Timing Template: [Aggressive, Normal, Polite, Stealth]
- Scope Filtering: [In-scope only, Include adjacent, Full discovery]
- Tool Selection: [Enable/disable specific scanners]
- Proxy Settings: [Direct, Burp Suite, Custom SOCKS]

1.7 6. Findings Management

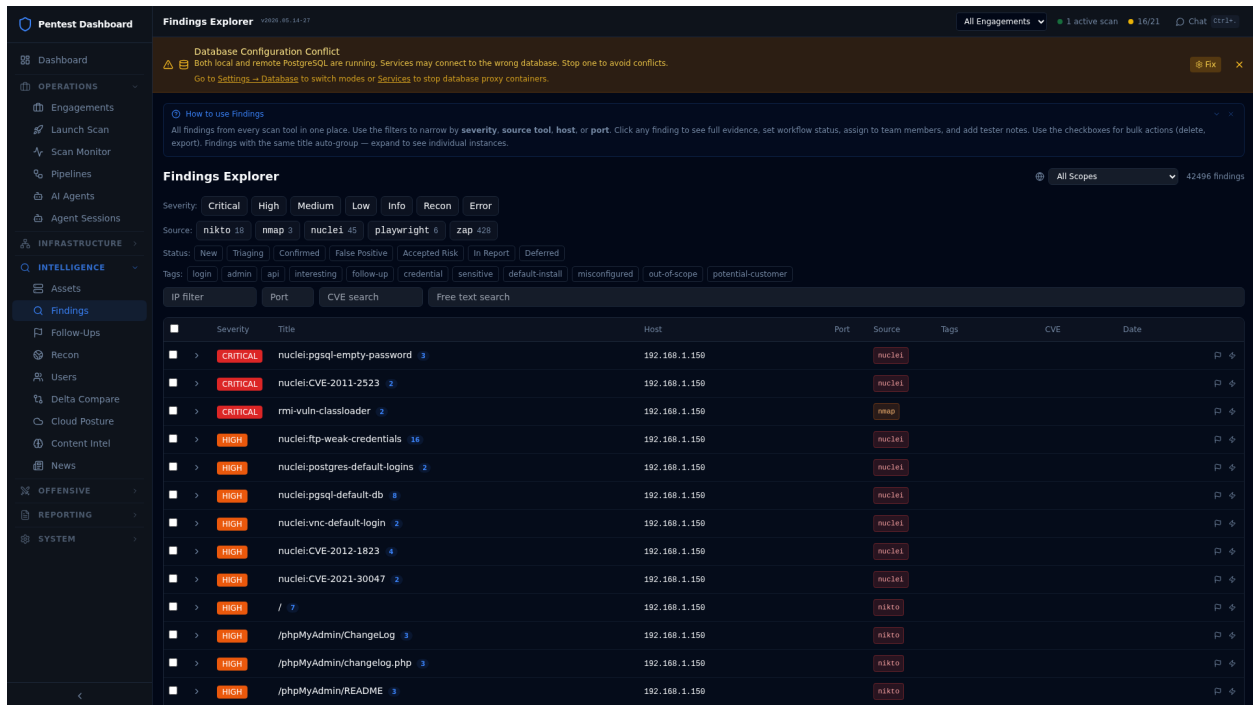


Figure 9: Findings Explorer

1.7.1 Step 1: Findings Explorer Interface

1.7.1.1 Comprehensive View Options

- [ANALYTICS] Summary Dashboard: High-level metrics and trend analysis
- ** Detailed Table **: Sortable, filterable findings with rich metadata
- ** Asset Map **: Network topology with vulnerability overlays
- ** Timeline View **: Chronological discovery and remediation tracking

1.7.2 Step 2: Finding Details & Evidence

1.7.2.1 Rich Finding Information

Finding Attributes:

Title & Description: Clear vulnerability summary

Severity Rating: Critical, High, Medium, Low, Info

CVSS Score: Industry-standard risk scoring

CWE/CVE References: Vulnerability classification

Affected Assets: Complete host and service details

Discovery Timeline: First seen, last verified dates

Figure 10: Findings Explorer

1.7.2.2 Evidence Documentation

- **** Screenshots****: Automated capture from web scanners
- **[LIST] Raw Output**: Complete tool output and command execution logs
- **[SEARCH] Proof of Concept**: Safe demonstration of vulnerability impact
- **[ANALYTICS] Exploitation Risk**: AI-assessed likelihood and business impact

1.7.3 Step 3: Workflow Management

1.7.3.1 Finding Status Tracking

- **** New****: Recently discovered, pending review
- **[SUCCESS] Confirmed**: Validated by security analyst
- **** False Positive****: Flagged as invalid finding
- **** Retest****: Scheduled for remediation validation
- **[SUCCESS] Fixed**: Confirmed remediation by client

1.7.3.2 Collaboration Features

- **** Comments & Notes****: Analyst discussion and client communication
 - **** Custom Tags****: Organizational categorization and workflow tracking
 - **** Notifications****: Automated alerts for critical findings
 - **** External Links****: Integration with ticketing and project management systems
-

1.8 7. Node & Tunnel Management

1.8.1 Step 1: Remote Access Overview

1.8.1.1 Supported Connection Types

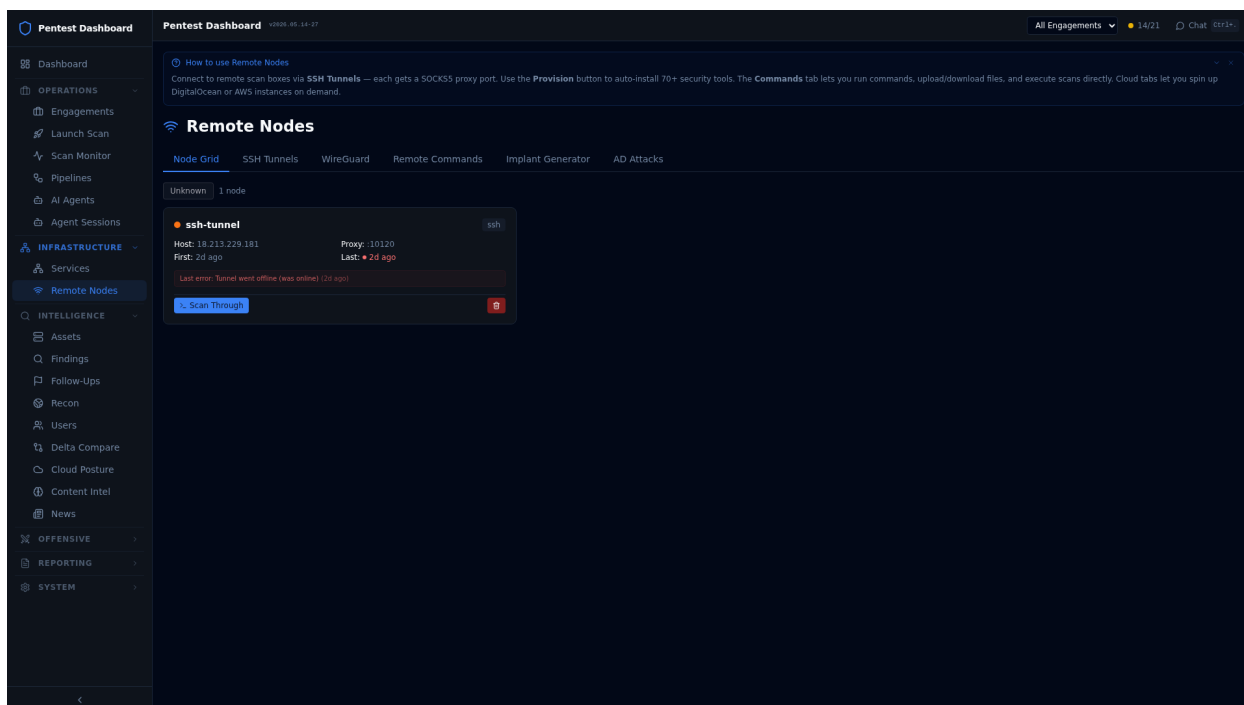


Figure 11: Nodes Overview

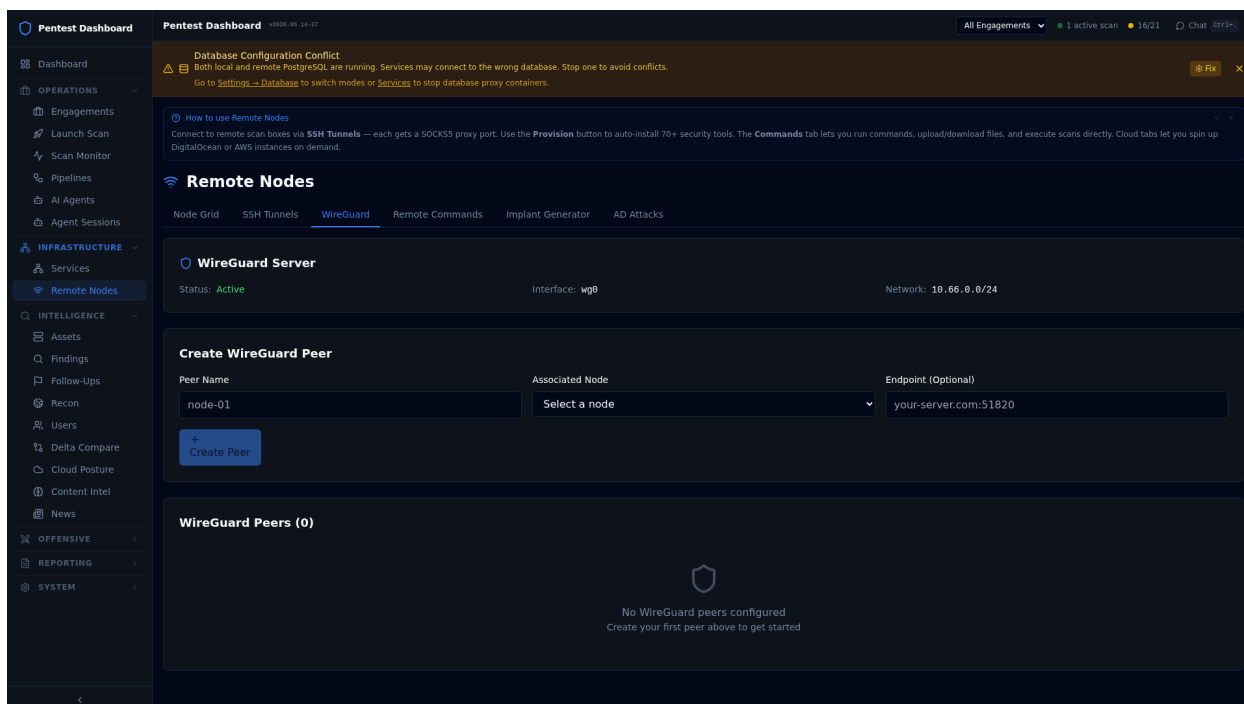


Figure 12: WireGuard Management

Figure 13: Nodes Overview

Figure 14: SSH Tunnels

Figure 15: WireGuard VPN

- **** SSH Tunnels****: Traditional encrypted tunneling with key-based authentication
- **** WireGuard VPN****: Modern, high-performance VPN tunneling
- **** C2 Frameworks****: Sliver implants for red team operations
- **[NETWORK] HTTP Proxies**: Burp Suite and ZAP integration for web testing

1.8.2 Step 2: WireGuard Management

1.8.2.1 Peer Configuration

WireGuard Setup:

1. **Generate Keypair**: Automatic server-side key generation
2. **IP Allocation**: Automatic subnet IP assignment
3. **Configuration Export**: Copy-paste or QR code for mobile apps
4. **Status Monitoring**: Real-time connection and traffic statistics

1.8.2.2 Mobile Device Support

- **** QR Code Generation****: Instant configuration for mobile WireGuard apps
- **[LIST] Configuration Export**: Text format for desktop clients
- **** Dynamic Reconfiguration****: Update peers without service restart
- **[ANALYTICS] Usage Statistics**: Bandwidth monitoring and connection analytics

1.8.3 Step 3: SSH Tunnel Operations

1.8.3.1 Tunnel Management

SSH Capabilities:

- SOCKS5 Proxy**: Dynamic port forwarding for tool traffic
- Local Forward**: Specific service access (RDP, VNC, etc.)
- Reverse Forward**: Callback channels for restricted environments

Auto-Reconnect: Resilient connections with health monitoring

1.8.3.2 Security Features

- **** Key-Based Authentication****: No password authentication allowed
- **[SECURE] Certificate Validation**: Host key verification and rotation
- **** Connection Monitoring****: Real-time status and failure alerting
- **[GRAPH] Performance Metrics**: Latency, bandwidth, and stability tracking

1.9 8. Reporting & Export

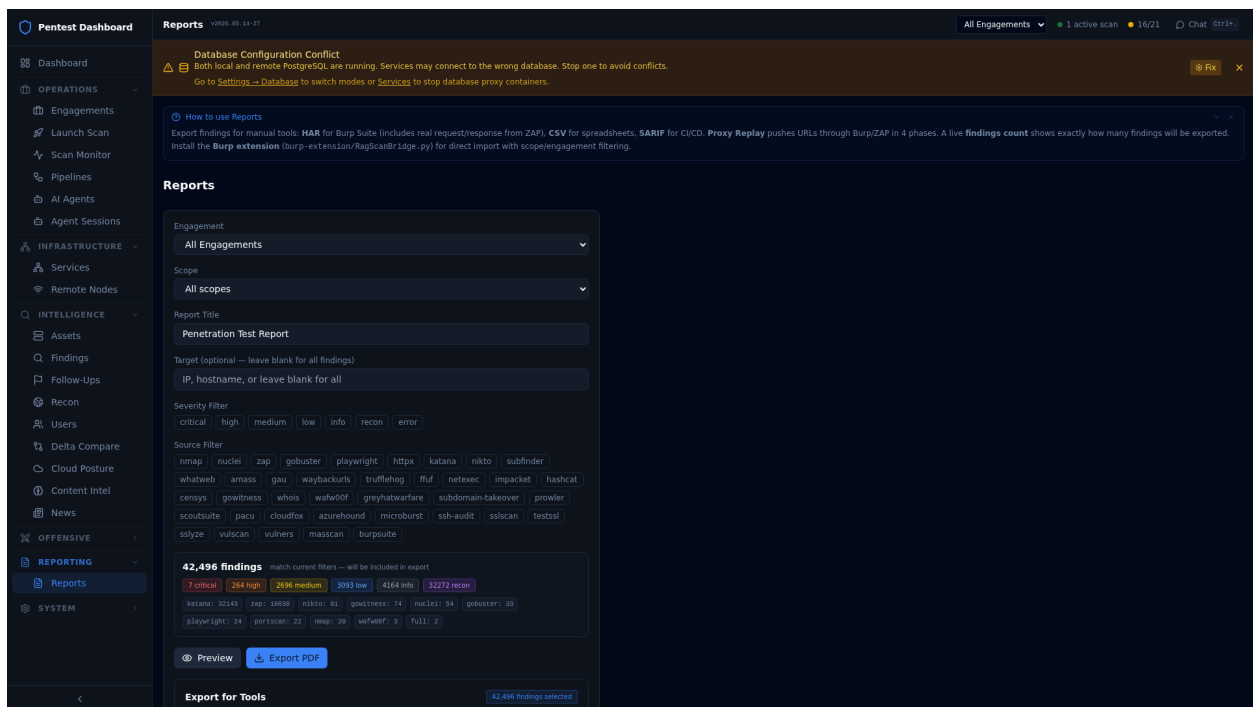


Figure 16: Reports Dashboard

1.9.1 Step 1: Report Generation Options

1.9.1.1 Standard Report Formats

Export Formats:

Executive Summary: High-level findings for leadership

Technical Report: Detailed findings for IT teams

SARIF: Industry-standard format for security tools

CSV: Spreadsheet analysis and custom processing

JSON: API integration and programmatic access

HAR: HTTP traffic for Burp Suite import

1.9.2 Step 2: Customizable Report Content

1.9.2.1 Report Sections

- **[LIST] Executive Summary**: Business impact and strategic recommendations
- **[TARGET] Scope Overview**: Tested systems and methodology summary

Figure 17: Reports Dashboard

- **[SEARCH] Findings Details:** Complete vulnerability documentation with evidence
- **[ANALYTICS] Risk Analysis:** Prioritized remediation roadmap
- **[GRAPH] Metrics & Trends:** Comparative analysis with previous assessments
- **** Remediation Guidance**:** Step-by-step resolution instructions

1.9.3 Step 3: Integration & Distribution

1.9.3.1 Automated Delivery

- **** Email Distribution**:** Scheduled delivery to stakeholder lists
 - **** API Webhooks**:** Real-time integration with external systems
 - **** Cloud Storage**:** Automatic upload to client document repositories
 - **** Mobile Notifications**:** Critical finding alerts for security teams
-

1.10 9. Advanced Features

1.10.1 Delta Analysis & Trending

- **** Comparative Analysis**:** Before/after remediation validation
- **[GRAPH] Risk Trends:** Improvement tracking over multiple assessments
- **** New Finding Detection**:** Automatic identification of environment changes
- **[ANALYTICS] Remediation Metrics:** Time-to-fix tracking and SLA monitoring

1.10.2 API Security Testing

- **** OpenAPI Import**:** Automatic test generation from Swagger/OpenAPI specs
- **[SEARCH] Endpoint Discovery:** Automated API surface mapping
- **** Authentication Testing**:** Token validation and session management

- **[ANALYTICS] Parameter Fuzzing:** Input validation and injection testing

1.10.3 Cloud Security Posture

- **** Multi-Cloud Support**:** AWS, Azure, GCP configuration assessment
- **** IAM Analysis**:** Permission auditing and privilege escalation detection
- **[PACKAGE] Resource Inventory:** Complete infrastructure mapping and compliance
- **** Misconfiguration Detection**:** Automated security best practice validation

1.10.4 Credential Testing & Management

- **** Secure Vault**:** Encrypted credential storage with audit logging
 - **[TARGET] Targeted Testing:** Username/password validation against discovered services
 - **[ANALYTICS] Success Tracking:** Credential reuse and policy compliance analysis
 - **[SECURE] Integration Security:** Safe handling of client authentication data
-

1.11 Next Steps

1.11.1 For Pentesters & Red Teams

1. **Start with Engagement Creation** - Define project scope and timeline
2. **Configure Target Scope** - Import and validate testing boundaries
3. **Launch AI Scanning Pipeline** - Automated discovery and vulnerability assessment
4. **Review & Validate Findings** - Manual verification and evidence collection
5. **Generate Client Reports** - Professional deliverables with remediation guidance

1.11.2 For Security Managers

1. **Review Dashboard Metrics** - Track team productivity and finding trends
2. **Monitor Service Health** - Ensure platform availability and performance
3. **Analyze Engagement ROI** - Measure testing effectiveness and coverage
4. **Plan Resource Allocation** - Scale infrastructure based on testing demand

1.11.3 Training & Support Resources

- **** Documentation Wiki**:** Comprehensive guides at `/about Documentation`
 - **** Video Tutorials**:** Step-by-step workflows and advanced techniques
 - **** API Reference**:** Complete OpenAPI documentation for automation
 - **** Community Support**:** Best practices sharing and troubleshooting assistance
-

1.12 Conclusion

The RAG Scan Stack provides a comprehensive, AI-enhanced platform for modern security testing workflows. By automating routine tasks, centralizing findings management, and providing advanced tunneling capabilities, teams can focus on high-value manual testing while maintaining comprehensive coverage of the attack surface.

Ready to start your first engagement? Begin with the Engagements module and follow this guide for a complete testing workflow. **# RAG Scan Stack - Management & Service Health Guide** **Operations, Monitoring & Administrative Tasks**

1.13 Table of Contents

1. Platform Administration Overview
 2. Service Health Monitoring
 3. Container & Service Management
 4. Performance Monitoring & Metrics
 5. Database Administration
 6. User Management & Security
 7. Backup & Recovery
 8. Troubleshooting Guide
 9. Maintenance Procedures
-

1.14 1. Platform Administration Overview

1.14.1 Administrative Responsibilities

- **[TOOL] Service Health Monitoring:** Ensure all components are operational
- **[ANALYTICS] Performance Management:** Monitor resource usage and optimize performance
- **** Security Administration**:** Manage access controls and audit logging
- **** Data Management**:** Backup, recovery, and data lifecycle policies
- **** Update Management**:** Apply security patches and feature updates
- **[GRAPH] Capacity Planning:** Scale resources based on testing demands

1.14.2 Admin Access Methods

Dashboard Admin Interface

`https://localhost:3002/settings` System Administration

Direct Container Access

`docker exec -it <container-name> /bin/bash`

Database Administration

`docker exec -it rag-postgres psql -U app -d scans`

Log Monitoring

`docker compose logs -f <service-name>`

1.15 2. Service Health Monitoring

1.15.1 Health Dashboard Overview

1.15.1.1 Real-Time Service Status Navigate to: **Dashboard Health Overview** or **Settings System Status**

Service Categories:

Core Services: Essential platform components

Scanner Services: Vulnerability scanning engines

AI Services: Machine learning and agent frameworks

Optional Services: Enhanced features and integrations

External Services: Third-party dependencies

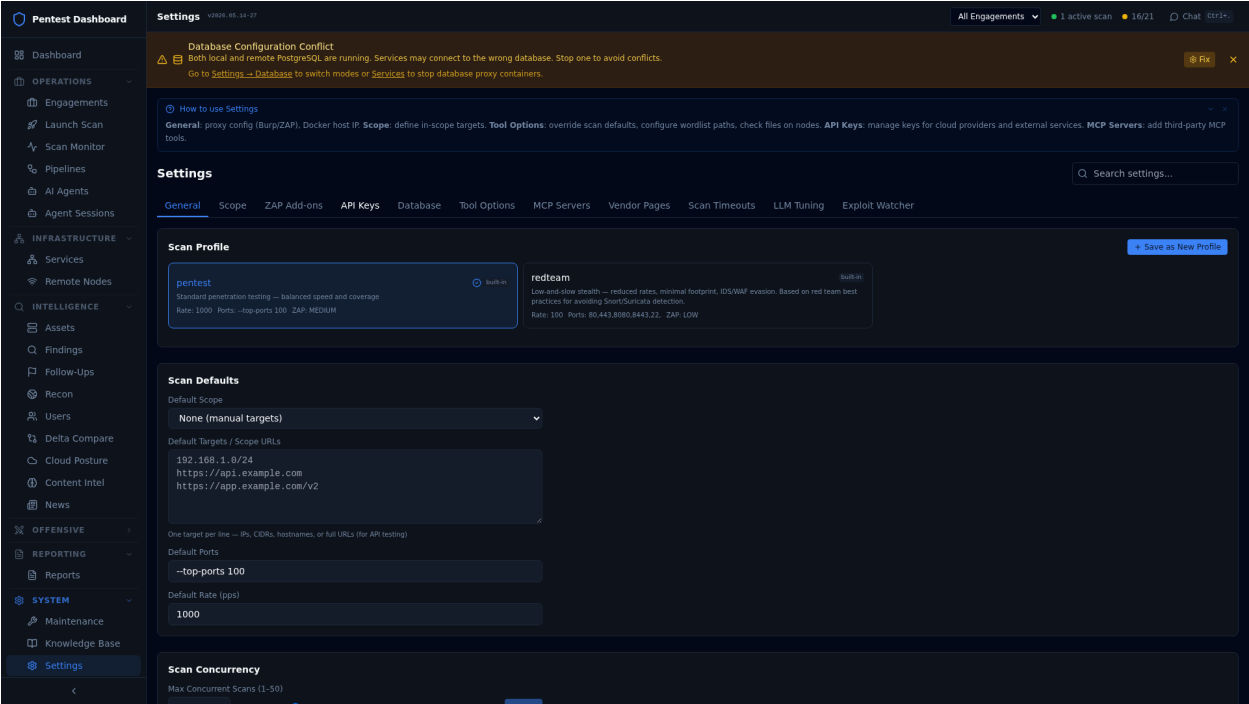


Figure 18: Health Dashboard

Figure 19: System Diagnostics

1.15.1.2 Service Health Indicators

Status Colors:

[SUCCESS] Healthy (Green): Service operational, all checks passing
[WARNING] Degraded (Yellow): Service functional with non-critical issues
[ERROR] Unhealthy (Red): Service down or critical failure
Stopped (Gray): Service intentionally disabled
Starting (Blue): Service initializing or restarting

1.15.2 Core Service Monitoring

1.15.2.1 Essential Services (Must be Healthy)

[SUCCESS] rag-api: Main API server and data coordination
[SUCCESS] pentest-dashboard: Web interface and BFF proxy
[SUCCESS] rag-postgres: Primary database (if using local DB)
[SUCCESS] container-logs: Service orchestration and Docker management
[SUCCESS] embedder: Text processing and AI feature extraction

1.15.2.2 Critical Health Checks

- **Database Connectivity:** Connection pool status and query response times
- **API Response Times:** Service latency and throughput metrics
- **Memory Usage:** Container resource consumption and limits
- **Disk Space:** Storage utilization and available capacity
- **Network Connectivity:** Inter-service communication status

1.15.3 Scanner Service Monitoring

1.15.3.1 Vulnerability Scanners

[SEARCH] Scanning Services:
[SUCCESS] nmap_scanner: Port discovery and service enumeration
[SUCCESS] nuclei-runner: Template-based vulnerability detection
[SUCCESS] web-scanner: Web application security testing
[SUCCESS] playwright-scanner: Modern JavaScript application testing
[SUCCESS] osint-runner: Open source intelligence gathering
[SUCCESS] pd-runner: Network discovery and fingerprinting
[SUCCESS] brutus-runner: Credential testing and brute force

1.15.3.2 Scanner Health Metrics

- **Queue Depth:** Pending scan jobs and processing capacity
 - **Tool Availability:** Scanner binary status and version checking
 - **Template Updates:** Nuclei template refresh and signature currency
 - **Resource Usage:** CPU and memory consumption during scans
 - **Error Rates:** Failed scan percentages and common failure modes
-

1.16 3. Container & Service Management

1.16.1 Service Control Interface

1.16.1.1 Navigate to: Settings Services

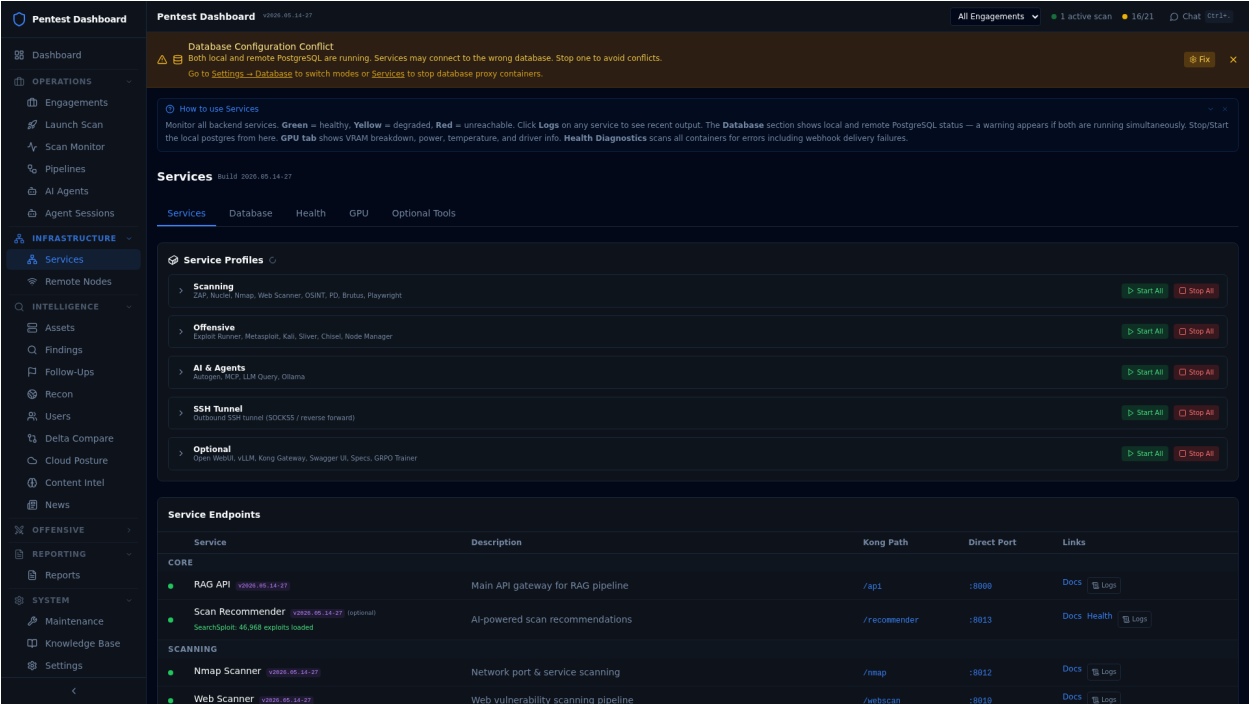


Figure 20: Services Management

Figure 21: Services Management

1.16.1.2 Profile-Based Service Groups

Service Profiles:

[TOOL] Core: Essential platform services (always running)
[SEARCH] Scan: Vulnerability scanning engines
AI: Machine learning and agent services
[TARGET] Offensive: Red team and exploitation tools
SSH-Tunnel: Remote access and tunneling
Optional: Enhanced features (GPU, Kong, etc.)

1.16.1.3 Service Control Operations

Available Actions:

-> Start Profile: Launch all services in a profile group
Stop Profile: Gracefully shutdown profile services
Restart Profile: Stop and restart all services
[TOOL] Start Individual: Launch specific container
Stop Individual: Stop specific container
[ANALYTICS] View Logs: Stream real-time container logs
[GRAPH] Resource Usage: Monitor CPU, memory, disk I/O

1.16.2 Container Management

1.16.2.1 Docker Compose Operations

Profile Management

```
docker compose --profile gpu up -d      # Start GPU services
docker compose --profile optional up -d  # Start optional services
docker compose down                      # Stop all services
```

Service-Specific Operations

```
docker compose restart rag-api          # Restart API server
docker compose logs -f nuclei-runner    # Stream scanner logs
docker compose ps                       # Show running containers
```

1.16.2.2 Individual Container Management

Container Operations

```
docker stop <container-name>           # Graceful shutdown
docker start <container-name>           # Start stopped container
docker restart <container-name>         # Stop and start
docker exec -it <container-name> bash   # Interactive shell
```

Resource Monitoring

```
docker stats                           # Real-time resource usage
docker inspect <container-name>        # Detailed container info
docker logs --tail 100 <container-name> # Recent log output
```

1.16.3 Health Check Configuration

1.16.3.1 Automated Health Monitoring

Health Check Intervals:

- API Services: 15-20 second intervals
- Scanners: 20-30 second intervals
- Database: 10 second intervals
- Optional Services: 30 second intervals

Failure Thresholds:

- Retries: 3-5 attempts before marking unhealthy
- Timeout: 5-10 seconds per health check
- Start Period: 30-120 seconds for initialization

1.17 4. Performance Monitoring & Metrics

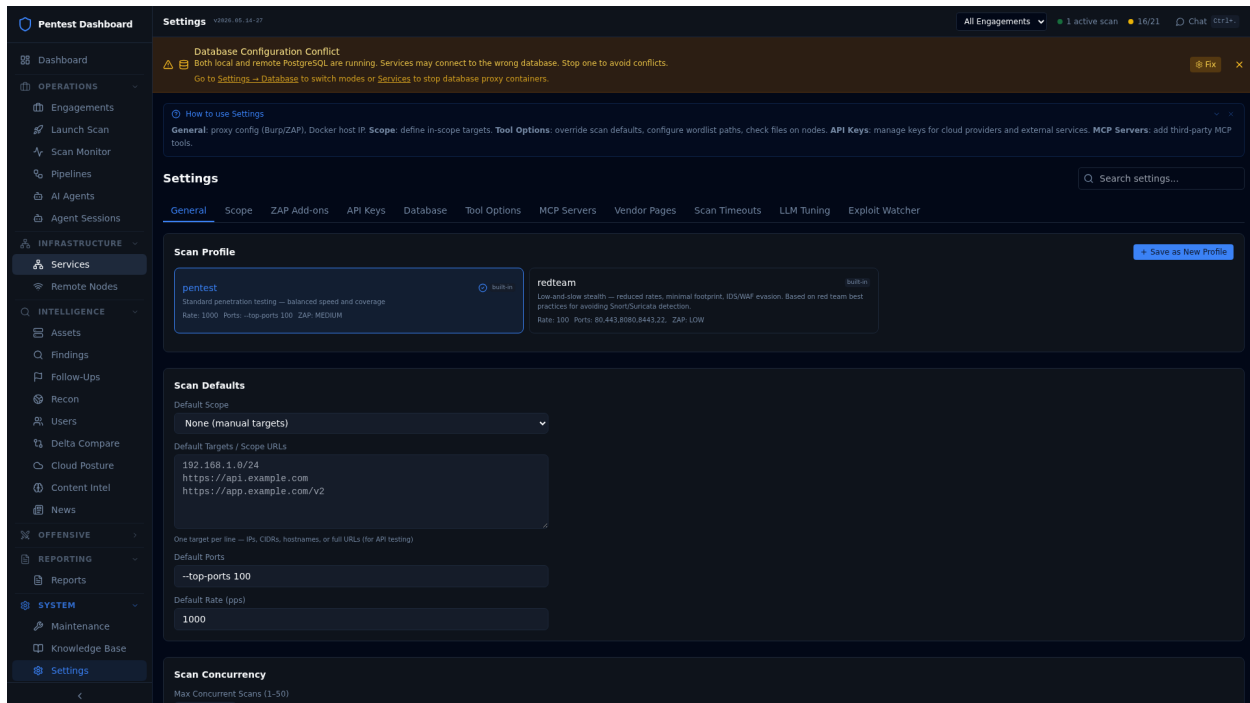


Figure 22: System Diagnostics

1.17.1 Resource Monitoring

1.17.1.1 System-Level Metrics

Host Resource Usage

```
htop          # Interactive process monitor
df -h         # Disk space usage
free -h       # Memory utilization
iostat        # Disk I/O statistics
```

1.17.1.2 Container Resource Monitoring

- **CPU Usage:** Per-container processor utilization
- **Memory Consumption:** RAM usage and swap activity
- **Disk I/O:** Read/write operations and throughput
- **Network Traffic:** Inter-service communication volume
- **Storage Usage:** Persistent volume utilization

Figure 23: OpSec Dashboard

1.17.2 Performance Dashboards

1.17.2.1 Navigate to: Settings Diagnostics

1.17.2.2 Key Performance Indicators

Platform KPIs:

[ANALYTICS] Scan Throughput: Targets processed per hour
Average Response Time: API latency percentiles
[TARGET] Success Rate: Successful scan completion percentage
Data Processing: Findings ingested per minute
Queue Performance: Job processing and wait times

1.17.2.3 Service-Specific Metrics

Per-Service Monitoring:

[SEARCH] Scanner Performance: Tool execution times and success rates
AI Processing: Model inference latency and accuracy
Database Performance: Query times and connection pooling
[NETWORK] Web Interface: Page load times and user interactions

1.17.3 Alerting & Notifications

1.17.3.1 Critical Alert Conditions

- **Service Downtime:** Core service failure or restart
- **Resource Exhaustion:** CPU, memory, or disk threshold exceeded
- **Database Issues:** Connection failures or slow queries
- **Security Events:** Authentication failures or unauthorized access
- **Scan Failures:** High error rates or tool unavailability

1.17.3.2 Alert Delivery Methods

- **Dashboard Notifications:** In-app alerts and status indicators
 - **Email Alerts:** Administrative notifications for critical issues
 - **Webhook Integration:** External monitoring system integration
 - **Log-Based Alerts:** Pattern matching in service logs
-

1.18 5. Database Administration

1.18.1 Database Overview

1.18.1.1 Supported Configurations

- **Local PostgreSQL:** Single-node database in `rag-postgres` container
- **Remote PostgreSQL:** External database with SSH tunnel support
- **High Availability:** Master/replica configurations for production
- **Cloud Managed:** AWS RDS, Azure Database, GCP Cloud SQL integration

1.18.2 Database Health Monitoring

1.18.2.1 Navigate to: Settings System Status Database Section

1.18.2.2 Critical Database Metrics

Database KPIs:

Active Connections: Current client connections vs. pool limits
[FAST] Query Performance: Average execution time and slow queries
Storage Usage: Database size growth and available space
Replication Status: Master/replica sync lag (if applicable)
[SECURE] Lock Contention: Blocking queries and deadlock frequency

1.18.2.3 Connection Pool Management

Pool Configuration:

- Max Connections: Typically 10-50 based on workload
- Idle Timeout: Automatic connection cleanup (300 seconds)
- Health Checks: Connection validation queries
- Overflow Handling: Queue management for peak demand

1.18.3 Database Maintenance

1.18.3.1 Regular Maintenance Tasks

-- *Connection Status*

```
SELECT count(*), state FROM pg_stat_activity GROUP BY state;
```

-- *Database Size*

```
SELECT pg_size_pretty(pg_database_size('scans'));
```

-- *Active Queries*

```
SELECT pid, username, application_name, state, query  
FROM pg_stat_activity WHERE state = 'active';
```

-- *Index Usage*

```
SELECT schemaname, tablename, indexname, idx_tup_read, idx_tup_fetch  
FROM pg_stat_user_indexes ORDER BY idx_tup_read DESC LIMIT 10;
```

1.18.3.2 Backup & Recovery

Database Backup

```
docker exec rag-postgres pg_dump -U app scans > backup.sql
```

Full System Backup

```
docker run --rm -v rag-pgdata:/data -v $(pwd):/backup ubuntu \
  tar czf /backup/database-backup.tar.gz /data
```

Recovery Operation

```
docker exec -i rag-postgres psql -U app scans < backup.sql
```

1.18.4 Schema Management

1.18.4.1 Migration Tracking

- **Version Control:** Database schema versioning with migration scripts
 - **Rollback Capability:** Safe schema changes with rollback procedures
 - **Testing:** Schema validation in development before production
 - **Documentation:** Change logs and impact assessment
-

1.19 6. User Management & Security

1.19.1 Authentication & Authorization

1.19.1.1 User Access Control

Authentication Methods:

API Key: Service-to-service authentication

Session-Based: Web interface user sessions

Certificate Auth: mTLS for service communication

OAuth Integration: Enterprise SSO (planned feature)

1.19.1.2 Role-Based Access Control

User Roles:

Administrator: Full system access and configuration

[TOOL] Operator: Scan execution and findings management

Viewer: Read-only access to findings and reports

Service Account: API-only access for integrations

1.19.2 Security Configuration

1.19.2.1 Navigate to: Settings Security

1.19.2.2 API Security Management

- **API Key Generation:** Secure key creation with expiration dates
- **Rate Limiting:** Request throttling to prevent abuse
- **IP Restrictions:** Source IP whitelist for sensitive operations
- **Audit Logging:** Complete access log with user attribution

1.19.2.3 Network Security

Security Controls:

[SECURE] TLS Encryption: All service communication encrypted

Container Isolation: Network segmentation between services
Firewall Rules: Minimal exposed ports and access control
Certificate Management: Automated SSL certificate rotation

1.19.3 Audit & Compliance

1.19.3.1 Security Monitoring

- **Access Logs:** User authentication and authorization events
- **Configuration Changes:** System modification tracking with rollback
- **Data Access:** Findings and sensitive data access logging
- **Failed Attempts:** Security event monitoring and alerting

1.19.3.2 Compliance Features

- **Data Retention:** Configurable retention policies for findings
 - **Export Controls:** Secure data export with audit trails
 - **Privacy Controls:** PII handling and data anonymization
 - **Regulatory Reporting:** Compliance report generation
-

1.20 7. Backup & Recovery

1.20.1 Backup Strategy

1.20.1.1 Automated Backup Components

Backup Scope:

Database: Full PostgreSQL backup with point-in-time recovery
Configuration: Service configurations and environment settings
Secrets: Encrypted backup of keys, certificates, and credentials
[ANALYTICS] Scan Data: Findings, reports, and evidence files
[GRAPH] Metrics: Historical performance and audit data

1.20.1.2 Backup Frequency & Retention

Backup Schedule:

Daily: Full database backup with 30-day retention
Hourly: Transaction log backup for point-in-time recovery
Weekly: Complete system backup with 90-day retention
Monthly: Archive backup with 1-year retention

1.20.2 Recovery Procedures

1.20.2.1 Recovery Scenarios

1. **Service Restart:** Container or service-level recovery
2. **Data Corruption:** Database rollback to known good state
3. **Complete System Failure:** Full platform restoration
4. **Partial Data Loss:** Selective recovery of specific components

1.20.2.2 Recovery Time Objectives

Recovery Metrics:

[TARGET] RTO (Recovery Time Objective): 2-4 hours for full system
[ANALYTICS] RPO (Recovery Point Objective): 1 hour maximum data loss

[FAST] Service Restart: 5-15 minutes for individual services
Database Recovery: 30-60 minutes for full restoration

1.20.3 Disaster Recovery

1.20.3.1 High Availability Configuration

- **Database Replication:** Master/standby configuration with automatic failover
 - **Load Balancing:** Distributed service deployment across multiple nodes
 - **Geographic Distribution:** Multi-site deployment for disaster recovery
 - **Cloud Integration:** Hybrid on-premise/cloud backup strategies
-

1.21 8. Troubleshooting Guide

1.21.1 Common Issues & Solutions

1.21.1.1 Service Startup Failures

Check container status

```
docker compose ps
```

View startup logs

```
docker compose logs <service-name>
```

Common solutions

```
docker compose pull # Update images
```

```
docker compose down && docker compose up -d # Full restart
```

```
docker system prune # Clean up resources
```

1.21.1.2 Database Connection Issues

Test database connectivity

```
docker exec rag-postgres pg_isready -U app -d scans
```

Check connection pool

```
docker exec rag-api python -c "  
from api import db  
print(db.engine.pool.status())  
"
```

Reset connections

```
docker compose restart rag-api rag-postgres
```

1.21.1.3 Scanner Tool Failures

Check tool availability

```
docker exec nuclei-runner nuclei -version
```

```
docker exec nmap_scanner nmap --version
```

Update scanner databases

```
docker exec nuclei-runner nuclei -update-templates
```

```
docker exec osint-runner /usr/bin/update-tools.sh
```

Clear scan queue

```
docker exec rag-api python -c "
from api import clear_scan_queue
clear_scan_queue()
"
```

1.21.2 Performance Issues

1.21.2.1 High Resource Usage

Identify resource-hungry containers
 docker stats --no-stream

Check system resources
 df -h # Disk space
 free -h # Memory usage
 top -p \$(pgrep -d',' -f docker) # Docker processes

1.21.2.2 Slow Response Times

1. **Database Performance:** Check slow queries and connection pool
2. **Network Latency:** Verify inter-service communication
3. **Resource Contention:** Monitor CPU and memory utilization
4. **Disk I/O:** Check storage performance and available space

1.21.3 Log Analysis

1.21.3.1 Centralized Log Collection

View all service logs
 docker compose logs --tail=100 -f

Service-specific logs
 docker compose logs rag-api --tail=50
 docker compose logs nuclei-runner --since=1h

Error pattern search
 docker compose logs | grep -i error
 docker compose logs | grep -i "failed\\|exception\\|timeout"

1.21.3.2 Log Level Configuration

- **DEBUG:** Detailed execution information for development
- **INFO:** Normal operation events and status updates
- **WARN:** Non-critical issues that may need attention
- **ERROR:** Service errors requiring immediate investigation
- **CRITICAL:** System failures requiring immediate response

1.22 9. Maintenance Procedures

1.22.1 Regular Maintenance Tasks

1.22.1.1 Daily Maintenance

Health check verification
 curl -k https://localhost:3002/api/health

```
# Database maintenance
docker exec rag-postgres psql -U app -d scans -c "VACUUM ANALYZE;"
```

```
# Log rotation
docker system prune --filter "until=24h"
```

```
# Resource monitoring
docker stats --no-stream | head -10
```

1.22.1.2 Weekly Maintenance

```
# Update scanner templates
docker exec nuclei-runner nuclei -update-templates
docker exec osint-runner /scripts/update-wordlists.sh
```

```
# Database optimization
docker exec rag-postgres psql -U app -d scans -c "REINDEX DATABASE scans;"
```

```
# Security scanning
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image --severity HIGH,CRITICAL rag-scan-stack-*
```

1.22.1.3 Monthly Maintenance

- **System Updates:** Apply security patches and minor updates
- **Capacity Planning:** Review resource usage trends and plan scaling
- **Security Audit:** Review access logs and security configurations
- **Backup Verification:** Test backup restore procedures
- **Performance Review:** Analyze metrics and optimize configurations

1.22.2 Update Management

1.22.2.1 Version Control Strategy

Update Categories:

```
[SECURE] Security Patches: Apply immediately for critical vulnerabilities
Bug Fixes: Apply during scheduled maintenance windows
Feature Updates: Plan deployment with stakeholder approval
[TOOL] Configuration Changes: Test in development before production
```

1.22.2.2 Update Procedure

1. **Backup Current State:** Full system backup before changes
2. **Update in Development:** Test updates in non-production environment
3. **Plan Maintenance Window:** Schedule downtime with stakeholders
4. **Apply Updates:** Execute changes with rollback capability
5. **Validate Functionality:** Comprehensive testing post-update
6. **Document Changes:** Update documentation and notify users

1.22.3 Monitoring & Alerting Maintenance

1.22.3.1 Alert Tuning

- **Threshold Adjustment:** Optimize alert thresholds based on historical data
- **False Positive Reduction:** Refine alert conditions to reduce noise
- **Escalation Procedures:** Update contact information and escalation paths
- **Integration Testing:** Verify external monitoring system integration

1.23 Emergency Procedures

1.23.1 Critical System Failure

1. **Immediate Response:** Assess scope of failure and potential causes
2. **Service Isolation:** Stop affected services to prevent cascade failures
3. **Stakeholder Notification:** Alert management and affected users
4. **Recovery Execution:** Follow documented recovery procedures
5. **Root Cause Analysis:** Post-incident investigation and remediation
6. **Process Improvement:** Update procedures based on lessons learned

1.23.2 Security Incident Response

1. **Incident Detection:** Identify and classify security events
 2. **Containment:** Isolate affected systems and limit damage
 3. **Evidence Preservation:** Secure logs and forensic evidence
 4. **Communication:** Notify security team and management
 5. **Recovery:** Restore systems to secure operational state
 6. **Post-Incident Review:** Analyze response and improve security controls
-

1.24 Conclusion

Effective management of the RAG Scan Stack requires proactive monitoring, regular maintenance, and rapid response to issues. This guide provides the foundation for maintaining a healthy, secure, and high-performing security testing platform.

Key Success Factors: - [SUCCESS] **Proactive Monitoring:** Continuous health checks and performance monitoring - [SUCCESS] **Automated Maintenance:** Scheduled tasks and automated recovery procedures - [SUCCESS] **Documentation:** Maintain current procedures and configuration documentation - [SUCCESS] **Training:** Ensure operational team understands platform architecture and procedures - [SUCCESS] **Testing:** Regular disaster recovery and incident response testing

For Additional Support: - ** Technical Documentation: **Platform architecture and API reference** - Troubleshooting Wiki: **Community-contributed solutions and best practices** - Emergency Contacts: **Escalation procedures for critical issues** - Operations Channel: **Team communication for coordination and knowledge sharing** # RAG Scan Stack - **Architecture Overview** Simple, API-Driven Security Testing Platform**

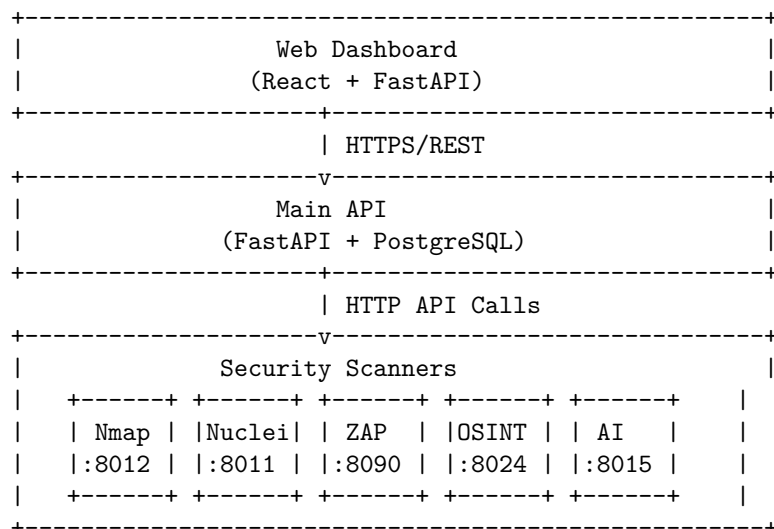
1.25 System Overview

RAG Scan Stack is a containerized security testing platform where **everything is accessible via REST APIs** and exposed through **MCP (Model Context Protocol)** tools for AI automation.

1.25.1 Core Principle

- **API-First Design:** Every function accessible via REST endpoints
 - **MCP Integration:** All capabilities exposed as AI-callable tools
 - **Container Architecture:** Each scanner runs in isolated Docker containers
 - **Unified Data Model:** All findings stored in PostgreSQL with consistent schema
-

1.26 Basic Architecture



1.27 Technology Stack

1.27.1 Core Components

- **Database:** PostgreSQL (findings, assets, configurations)
- **API Server:** FastAPI (main coordination and data management)
- **Web Interface:** React + TypeScript (dashboard for manual use)
- **Container Runtime:** Docker + Docker Compose

1.27.2 Security Scanners (Each in separate container)

- **Nmap** (Port scanning): `nmap_scanner:8012`
 - **Nuclei** (Vulnerability scanning): `nuclei-runner:8011`
 - **ZAP** (Web application scanning): `zap:8090`
 - **OSINT** (Reconnaissance): `osint-runner:8024`
 - **Playwright** (Modern web apps): `playwright-scanner:8014`
 - **Brutus** (Credential testing): `brutus-runner:8026`
-

1.28 API-First Design

1.28.1 Everything Has an API Endpoint

Scan Management:

```
POST /run_masscan_nmap      # Start port scan
POST /jobs/masscan-nmap/upload # Upload scan results
GET  /scans/{id}           # Check scan status
POST /ingest/nmap           # Process scan results
POST /ingest/nuclei         # Process vuln scan results
```

Finding Management:

```
GET    /assets              # List discovered assets
```

```
GET    /credentials          # List credential findings
DELETE /findings/bulk     # Bulk delete findings
GET    /ports/open         # List open ports
```

Export Formats:

```
GET    /export/sarif        # SARIF v2.1.0 export
GET    /export/har        # HAR format export
GET    /export/burp       # Burp Suite XML export
POST   /findings/export   # Custom export formats
```

1.28.2 Direct Scanner APIs

Each scanner exposes its own REST API: - https://nmap_scanner:8012/ - Port discovery service - <https://nuclei-runner:8011/> - Vulnerability testing service - <https://zap:8090/> - Web application scanning - <https://osint-runner:8024/> - Reconnaissance tools

1.29 MCP Tool Integration

1.29.1 What is MCP?

Model Context Protocol - allows AI agents to call platform functions as “tools”

1.29.2 Available MCP Tools (26 total across 6 servers)

Scanning Tools: - `scan_ports()` - Run Nmap port scans - `scan_vulnerabilities()` - Run Nuclei templates - `scan_web_app()` - ZAP web application testing - `scan_subdomains()` - OSINT reconnaissance

Data Management: - `search_findings()` - Query vulnerability database - `get_asset_info()` - Asset and service details - `export_results()` - Generate reports

System Control: - `check_service_health()` - Monitor scanner status - `manage_scan_queue()` - Control scan execution

1.29.3 MCP Server Endpoints

```
http://localhost:9016 # pentest-sessions (scan coordination)
http://localhost:9017 # pentest-scanning (nmap, nuclei tools)
http://localhost:9018 # pentest-recon (OSINT, subdomain enum)
http://localhost:9019 # pentest-exploit (exploit execution)
http://localhost:9020 # pentest-credentials (credential testing)
http://localhost:9021 # scan-pipelines (automated workflows)
```

1.30 Data Flow

1.30.1 Simple Scan Workflow

1. **API Request** POST `/run_masscan_nmap` with target IPs
2. **Container Execution** Nmap scanner container runs scan
3. **Result Processing** ETL parser normalizes Nmap XML output
4. **Database Storage** Findings stored in PostgreSQL via POST `/ingest/nmap`
5. **API Response** Results available via GET `/assets` and GET `/ports/open`

1.30.2 Database Schema (Simplified)

```
-- Core Tables
assets          # Target IPs and hostnames
ports           # Discovered services
vulns           # Vulnerability findings
web_findings    # Web application issues
recon_findings  # OSINT discoveries

-- Metadata
scan_runs       # Scan execution records
engagements     # Project organization
```

1.31 Remote Access & Tunneling

1.31.1 SSH Tunnels

- **Purpose:** Route scans through compromised hosts
- **Implementation:** autossh containers with SOCKS5 proxies
- **API:** POST /ssh/connect with credentials

1.31.2 WireGuard VPN

- **Purpose:** Modern VPN alternative to SSH
 - **Implementation:** WireGuard server container
 - **API:** POST /wg/peers to create VPN configs
-

1.32 Deployment

1.32.1 Single Host Deployment

```
git clone <repository>
docker compose up -d
curl https://localhost:3002 # Web interface (HTTPS)
curl https://localhost:8000/health # API health check
```

1.32.2 Environment Requirements

- **Minimum:** 8GB RAM, 4 CPU cores, 100GB storage
 - **Recommended:** 16GB RAM, 8 CPU cores, 500GB SSD
 - **Network:** Internet access for scanner updates
 - **Optional:** NVIDIA GPU for AI features
-

1.33 Security Model

1.33.1 Container Isolation

- Each scanner runs in separate container
- Minimal privileges (no root where possible)
- Network segmentation via Docker networks

1.33.2 API Security

- API key authentication
- HTTPS encryption for all communication
- Rate limiting and request validation

1.33.3 Data Protection

- Database encryption at rest
 - Secure credential storage
 - No plaintext secrets in containers
-

1.34 Integration Points

1.34.1 Import/Export

- **Nessus:** Import .nessus files via POST `/ingest/nessus`
- **Burp Suite:** Export to XML sitemap via GET `/export/burp`
- **HAR:** HTTP Archive format export via GET `/export/har`
- **SARIF:** Security standard export via GET `/export/sarif`

1.34.2 External APIs

- **SHODAN:** Internet-wide scanning data
 - **VirusTotal:** File and URL analysis
 - **CVE Databases:** Vulnerability intelligence
-

1.35 Monitoring & Health

1.35.1 Service Health

- **Health Endpoint:** GET `/health` - All service status
- **Individual Services:** Each scanner has `/health` endpoint
- **Database:** Connection pool and query performance monitoring

1.35.2 Logging

- **Container Logs:** `docker compose logs <service>`
 - **Application Logs:** Structured JSON logging
 - **Audit Trail:** All API calls logged with user attribution
-

1.36 Summary

RAG Scan Stack is fundamentally:

1. **API-Driven:** Every function accessible via REST endpoints
2. **Container-Based:** Isolated, scalable scanner services
3. **MCP-Enabled:** AI agents can call any platform function
4. **Data-Centric:** Unified PostgreSQL storage with consistent schema
5. **Integration-Ready:** Standard formats for import/export

The architecture prioritizes simplicity and API accessibility over complexity.

This architecture enables both manual security testing via the web interface and fully automated testing via AI agents using the MCP tools.